

剖析 Agent 运行框架

The Anatomy of an Agent Harness

LangChain · Vivek Trivedy · 2026-03-11 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网站导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

核心要点

- **拆解复杂目标**：规划工具让 Agent 能够分解任务、跟踪进度，并随着新认识的积累调整行动。
- **并行委派工作**：为独立子任务启动子 Agent，每个子 Agent 都有隔离的上下文。

简而言之：Agent = 模型 + 运行框架。运行框架工程，就是围绕模型构建系统，让模型成为能够完成工作的引擎。模型承载智能，运行框架让这种智能发挥作用。本文将定义什么是运行框架，并推导出当今和未来的 Agent 所需的核心组件。

谁能给“运行框架”下个定义？

Agent = 模型 + 运行框架

只要不是模型本身，就属于运行框架。

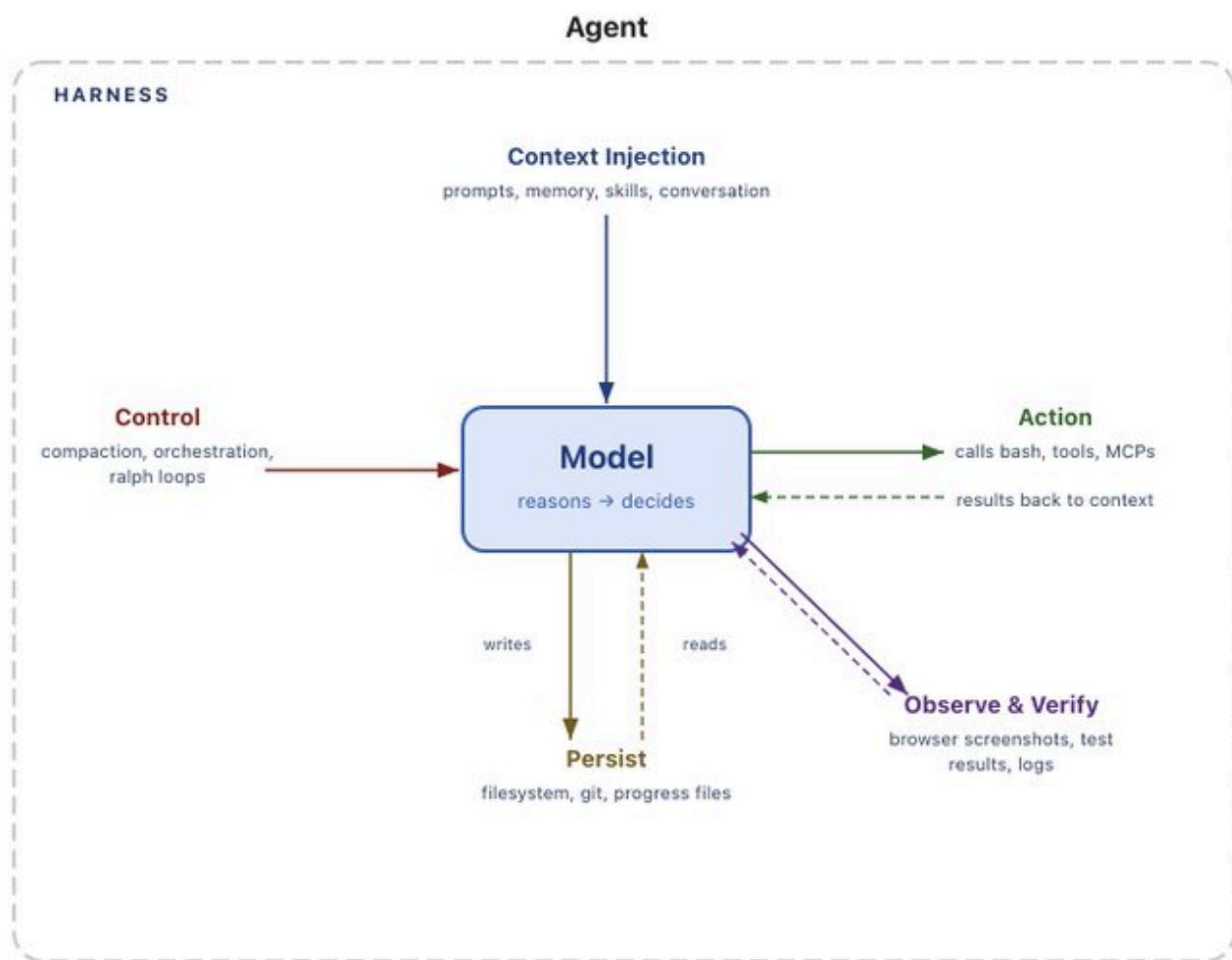
运行框架，是模型本身之外的所有代码、配置和执行逻辑。裸模型并不是 Agent。但当运行框架为它提供状态、工具执行、反馈循环以及可强制落实的约束等能力后，它就成为了 Agent。

具体来说，运行框架包含以下内容：

- 系统提示词
- 工具、Skills、MCP 及其描述
- 配套基础设施（文件系统、沙箱、浏览器）
- 编排逻辑（启动子 Agent、任务交接、模型路由）
- 用于确定性执行的钩子 / 中间件（上下文压缩、继续执行、静态代码检查）

Agent 系统中，模型与运行框架的边界可以有很多种划分方式，而且往往很混乱。但在在我看来，上面的定义最清晰，因为它迫使我们思考如何围绕模型的智能设计系统。

接下来，本文将从模型这一核心基础单元出发，逐一介绍运行框架的核心组件，并反向推导每个部分为什么存在。



从模型的视角看：为什么需要运行框架

我们希望 **Agent** 完成的一些事情，模型并不能开箱即用做到。这就是运行框架发挥作用的地方。模型（大多）接收文本、图像、音频、视频等数据，然后输出文本。仅此而已。模型本身无法直接做到：

- 在多次交互之间维持持久状态
- 执行代码
- 获取实时知识
- 为完成工作而搭建环境、安装软件包

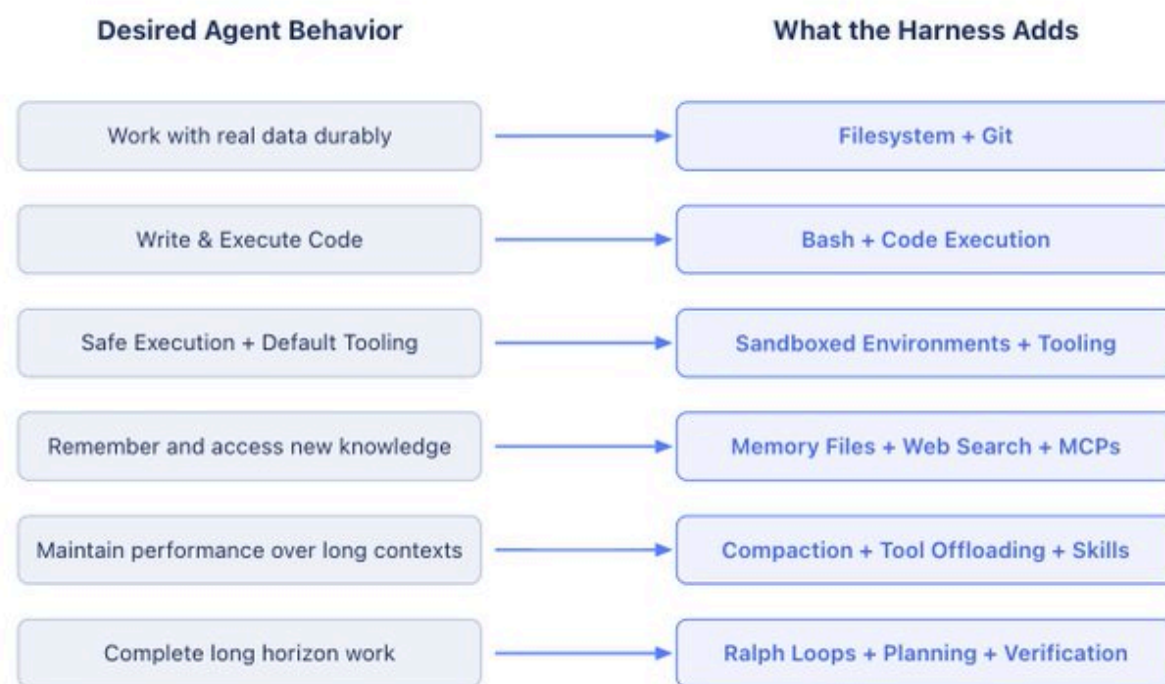
这些都是**运行框架层面的功能**。LLM 的结构决定了，要让它们完成有用的工作，就需要某种包裹在外部的机制。例如，要实现“聊天”这样的产品体验，我们就把模型放进一个 while 循环，记录之前的消息，再追加用户的新消息。读到这里的每个人都已经使用过这类运行框架。核心思路是：将我们期望的 **Agent** 行为，转化为运行框架中实际存在的功能。

从期望的 Agent 行为反推运行框架工程

运行框架工程让人类能够注入有用的先验知识，引导 Agent 的行为。随着模型能力增强，人们通过运行框架有针对性地扩展和修正模型，让其完成过去无法完成的任务。

我们不会穷举运行框架的所有功能。我们的目标是以“帮助模型完成有用的工作”为出发点，推导出一组功能。我们将遵循这样的模式：

期望实现（或修正）的行为 → 帮助模型实现这种行为的运行框架设计。



Each harness feature derives from a behavior the model can't deliver on its own

用文件系统实现持久存储与上下文管理

我们希望 Agent 拥有持久存储，以便与真实数据交互，将上下文容纳不下的信息移出，并在不同会话之间保存工作成果。

模型只能直接处理上下文窗口中的知识。没有文件系统时，用户必须把内容直接复制粘贴给模型。这种用户体验很笨拙，也不适用于自主 Agent。现实世界早已使用文件系统开展工作，因此模型自然也在数十亿 token 的相关使用资料上接受过训练。顺理成章的解决方案是：

运行框架内置文件系统抽象，以及执行文件系统操作的工具。

文件系统可以说是运行框架最基础的构件，因为它带来了以下能力：

- Agent 拥有一个工作区，可以读取数据、代码和文档。
- 工作内容可以逐步增添并移出上下文，不必把一切都放在上下文中。Agent 可以存储中间输出，并维持超越单次会话的状态。

- **文件系统天然适合协作。**多个 Agent 与人类可以通过共享文件协调工作。Agent Teams 等架构就依赖这一点。

Git 为文件系统增加了版本管理，使 Agent 能够跟踪工作、回滚错误，并通过分支进行实验。下文还会再次谈到文件系统，因为它也是实现其他所需功能的重要基础构件。

将 Bash 与代码作为通用工具

我们希望 Agent 自主解决问题，而不需要人类预先设计每一种工具。

目前 Agent 的主要执行模式是 ReAct 循环：模型进行推理，通过工具调用采取行动，观察结果，然后在 while 循环中重复这一过程。但运行框架只能执行它已具备相应逻辑的工具。与其要求用户为每一种可能的行动构建工具，更好的办法是给 Agent 提供 Bash 这样的通用工具。

运行框架内置 Bash 工具，让模型能够通过编写和执行代码来自主解决问题。

Bash 加上代码执行能力，是朝着给模型一台计算机、让它自主弄清其余事情迈出的一大步。模型可以通过代码即时设计自己的工具，而不再受限于一组预先配置好的固定工具。

运行框架仍然会内置其他工具，但代码执行已经成为自主解决问题时默认采用的通用策略。

用沙箱和工具执行、验证工作

Agent 需要一个默认配置合理的环境，才能安全地采取行动、观察结果并推进工作。

我们给了模型存储和执行代码的能力，但这些操作总得在某个地方进行。在本地运行 Agent 生成的代码存在风险，而且单一本地环境无法扩展到大规模的 Agent 工作负载。

沙箱为 Agent 提供安全的操作环境。运行框架可以连接到沙箱，在其中运行代码、检查文件、安装依赖并完成任务，而不是在本地执行。这实现了安全、隔离的代码执行。为了进一步提高安全性，运行框架可以设置命令允许列表，并强制实施网络隔离。沙箱也带来了扩展能力：环境可以按需创建，为大量任务分别分配，并在工作完成后销毁。

好的环境具备良好的默认工具配置。运行框架负责配置工具，使 Agent 能够完成有用的工作。这包括预装语言运行时和软件包、用于 Git 与测试的命令行工具，以及用于网页交互和验证的浏览器。

浏览器、日志、截图和测试运行器等工具，为 Agent 提供了观察与分析自身工作的手段。这有助于它们建立**自我验证循环，在其中编写应用代码、运行测试、检查日志并修复错误。**

模型本身并不会自动配置执行环境。Agent 在哪里运行、有哪些工具可用、能访问什么，以及如何验证自己的工作，都是运行框架层面的设计决策。

通过记忆与搜索实现持续学习

Agent 应当记住已经见过的内容，并能够获取训练时尚不存在的信息。

除了权重中蕴含的知识，以及当前上下文中的内容，模型没有其他知识。在无法修改模型权重的情况下，“增加知识”的唯一方式就是注入上下文。

在记忆方面，文件系统再次成为核心构件。运行框架支持 AGENTS.md 等记忆文件规范，并在 Agent 启动时将其注入上下文。当 Agent 增添和编辑这些文件后，运行框架会把更新后的文件加载到上下文中。这是一种持续学习：Agent 持久保存某次会话中获得的知识，并将其注入后续会话。

知识截止日期意味着，如果用户不直接提供新数据，模型就无法直接获取新版软件库等信息。为了获得最新知识，网页搜索以及 Context7 等 MCP 工具，可以帮助 Agent 访问知识截止日期之后的信息，例如新版软件库，或者训练结束时尚不存在的当前数据。

网页搜索以及用于查询最新上下文的工具，是值得内置到运行框架中的基础能力。

应对上下文退化

Agent 的表现不应随着工作推进而恶化。

上下文退化（Context Rot）描述的是：随着上下文窗口逐渐填满，模型的推理和任务完成能力会变差。上下文是宝贵而稀缺的资源，因此运行框架需要相应的管理策略。

如今，运行框架在很大程度上是落实良好上下文工程的机制。

上下文压缩解决的是上下文窗口接近填满时该怎么办的问题。如果没有压缩，当对话超过上下文窗口容量时会发生什么？一种情况是 API 报错，这显然不理想。运行框架必须采取某种策略处理这一情形。因此，压缩会智能地将现有上下文中的内容移出并加以总结，让 Agent 能够继续工作。

工具调用输出卸载有助于减轻大型工具输出的影响：它们可能给上下文窗口塞入大量噪声，却没有提供有用信息。当工具输出超过某个 token 数量阈值时，运行框架只保留开头和结尾的 token，把完整输出移到文件系统中，供模型在需要时访问。

Skills解决的是 Agent 启动时加载到上下文中的工具或 MCP 服务器过多的问题。这会导致 Agent 还没开始工作，表现就已经下降。Skills 是运行框架层面的基础构件，通过**渐进式披露**解决这个问题。模型并没有选择在启动时将 Skill 的前置信息加载到上下文中，但运行框架可以支持这种做法，保护模型免受上下文退化的影响。

长时间跨度的自主执行

我们希望 Agent 能够在很长的时间跨度内，自主、正确地完成复杂工作。

自主创建软件，是编程 Agent 追求的终极目标。但今天的模型仍会过早停止，难以分解复杂问题，而且当工作跨越多个上下文窗口时，表现会不连贯。优秀的运行框架必须针对这些问题进行设计。

前面介绍的运行框架基础能力，在这里开始相互叠加。长时间跨度的工作需要持久状态、规划、观察和验证，才能跨越多个上下文窗口持续推进。

通过文件系统和 Git 跨会话跟踪工作。Agent 在长期任务中会产生数百万 token，因此文件系统需要持久保存工作成果，以便持续跟踪进度。加入 Git 后，新的 Agent 可以迅速了解项目的最新进展与历史。多个 Agent 一起工作时，文件系统也充当共享工作记录，让它们能够相互协作。

通过 Ralph 循环继续工作。Ralph 循环是一种运行框架模式：它通过钩子拦截模型尝试退出的行为，并在一个干净的上下文窗口中重新注入原始提示词，迫使 Agent 继续朝完成目标推进。文件系统让这一机制成为可能，因为每次迭代都从全新的上下文开始，却会读取上一次迭代留下的状态。

通过规划和自我验证保持方向。规划就是模型将目标分解为一系列步骤。运行框架通过良好的提示词，以及注入有关如何使用文件系统中计划文件的提醒来支持这一过程。完成每一步后，Agent 都可以通过自我验证检查工作的正确性。运行框架中的钩子可以运行预先定义的测试套件，并在失败时把错误消息反馈给模型；也可以提示模型独立评估自己的代码。验证通过测试为解决方案提供依据，并产生用于自我改进的反馈信号。

运行框架的未来

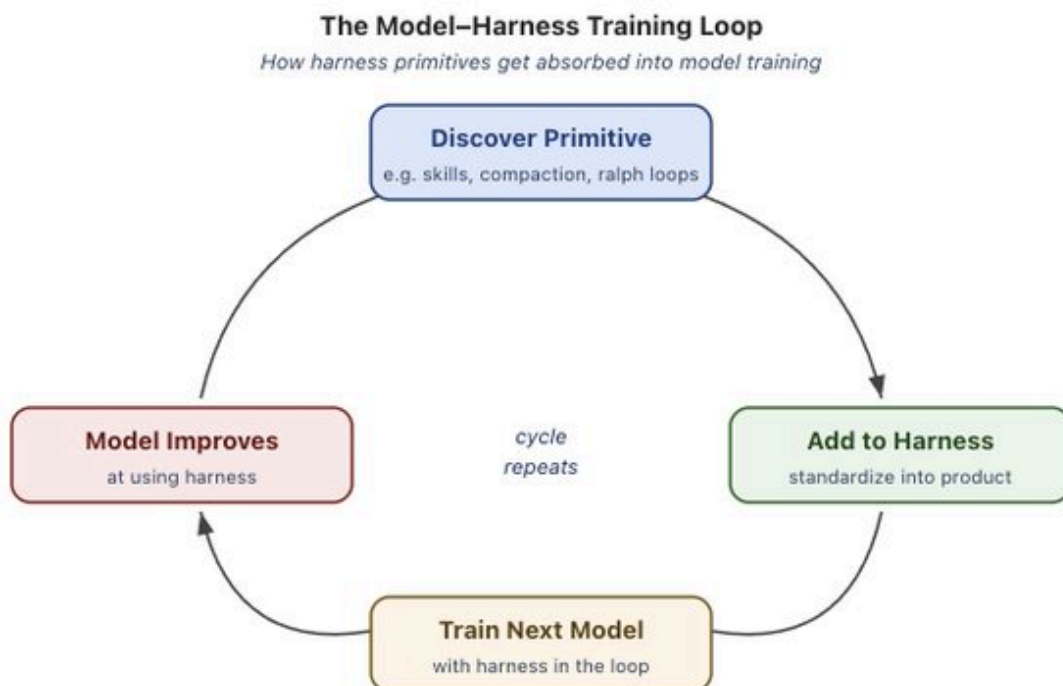
模型训练与运行框架设计的耦合

如今，Claude Code、Codex 等 Agent 产品的后训练过程将模型和运行框架一起纳入循环。这有助于模型改进运行框架设计者认为它们应当原生擅长的操作，例如文件系统操作、执行 Bash、规划，或者通过子 Agent 并行开展工作。

这形成了一个反馈循环：人们发现有用的基础能力，将其加入运行框架，再在训练下一代模型时使用它们。随着这个循环不断重复，模型在其受训所用的运行框架中会变得更有能力。

但这种共同演化也会对泛化产生有趣的副作用。例如，改变工具逻辑可能导致模型表现变差。Codex-5.3 提示词指南中的这个例子就描述了用于编辑文件的 `apply_patch` 工具逻辑。真正智能的模型，在不同补丁方法之间切换时本应几乎不会遇到困难，但将运行框架纳入训练循环，会造成这种过拟合。

但这并不意味着，最适合你所做任务的运行框架，就是模型后训练时使用的那一个。Terminal Bench 2.0 排行榜就是一个很好的例子。Claude Code 中的 Opus 4.6 得分，远低于其他运行框架中的 Opus 4.6。我们此前的一篇博客展示过：仅仅改变运行框架，就把编程 Agent 在 Terminal Bench 2.0 上的名次从前 30 提升到了前 5。针对你的任务优化运行框架，仍有很大的潜力可以挖掘。



运行框架工程将走向何方

随着模型能力增强，今天由运行框架承担的一部分能力会被模型吸收。例如，模型会原生地更擅长规划、自我验证，以及在长时间跨度上保持连贯，从而减少对上下文注入的需求。

这似乎意味着，运行框架的重要性会逐渐下降。但正如提示词工程至今仍有价值一样，运行框架工程很可能也会继续在构建优秀 Agent 时发挥作用。

运行框架如今确实会弥补模型的不足，但它也围绕模型智能构建系统，使其更加有效。无论模型基础智能水平如何，配置良好的环境、合适的工具、持久状态与验证循环，都能提高模型的效率。

运行框架工程是一个十分活跃的研究领域。在 LangChain，我们用这些研究来改进运行框架构建库 deepagents。以下是我们目前正在探索的几个尚未解决、又很有趣的问题：

- 编排数百个 Agent，在同一个共享代码库中并行工作
- 让 Agent 分析自身的执行轨迹，识别并修复运行框架层面的失败模式
- 让运行框架针对具体任务，按需及时、动态地组装合适的工具和上下文，而非预先固定配置

本文尝试定义什么是运行框架，以及我们希望模型完成的工作如何塑造运行框架。

模型承载智能，运行框架则是让这种智能发挥作用的系统。

愿我们构建更多运行框架、更好的系统，以及更好的 Agent。

通过 **LangSmith Deployment** 一键部署 Deep Agents。与我们团队的专家交流。